

William Huang PSET 1

```
In [12]: # First import packages
from acoli import acoli_hist
import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d import Axes3D
import pandas as pd
from PIL import Image
```

Question 1

Question 1a.

In a function of two variables, $f(x, y)$, the function dictates a 3D surface. Therefore, at any given point on the surface, there exists a tangent plane. Since the tangent plane has an infinite number of lines, where every line is tangent to the point on the surface, there are an **infinite number of tangent lines**.

Question 1b.

Given a surface with points $(x, y, f(x, y))$, the gradient is computed by

$$\left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$$

The gradient is the rate of change of a function. In other words, the gradient is computed by taking the partial derivative of the function, $f(x, y)$, with respect to the variables, x and y . This would give us $\frac{\partial f}{\partial x}$ (rate of change wrt x -direction) and $\frac{\partial f}{\partial y}$ (rate of change wrt y -direction). Using these partial derivatives, we can form a vector that would dictate the rate of change in the x and y directions. As such, it is a 2-dimensional vector.

To plot it into a 3-dimensional vector, you plot it using the coordinates

$$\left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, 0 \right)$$

This is because the function, $f(x, y)$ only changes with changes to the variables x and y . Since the z -coordinate in this example, $f(x, y)$, does not change implicitly itself, it will not change beyond changes in x and y . In other words, because the function is not dependent on z , the gradient of the function, or the direction of steepest ascent, will not be affected by the z -values. Therefore, the partial derivative of the function with respect to z will always be 0.

Question 1c.

The directional derivative is a scalar that dictates the rate of change of a function in a certain direction.

Formally, the directional derivative is

$$D_u f(x, y) = \nabla f(x, y) \cdot u$$

This formula states that the directional derivative of a function, f , at a point (x, y) , is dependent on the gradient of the function at the point and a unit direction vector u . In other words, the directional derivative finds the rate of change by finding how much the direction, u , deviates from the gradient. At its maximum, the directional derivative can be the magnitude of the gradient.

Question 1d.

The gradient always points to the direction of steepest ascent. Since you are climbing a mountain, the goal is to reach the top of the mountain, or the point of highest elevation. If you deviate from the gradient, any step taken not in the direction of the gradient will get you to a point that is not as high as you would have gone in the direction of the gradient. Therefore, you will have to take additional steps to reach the same elevation had you gone in the gradient's direction. Because you are taking additional steps after deviating from the gradient, it will lead to a longer path.

Question 2

Question 2a.

```
In [13]: initpos = [0, 0]
         foodfn = 2
         rttp = 0.5
         ttrp = 0.5
         ntrials = 2
         nsteps = 10
         nbins = 50
         visualize=False
         verbose=False

         X = np.linspace(0, nbins, nbins)
         Y = np.linspace(0, nbins, nbins)

         histm, food = acol_hist(initpos, foodfn, rttp, ttrp, ntrials, nsteps, nbins)

         # Heat Map
```

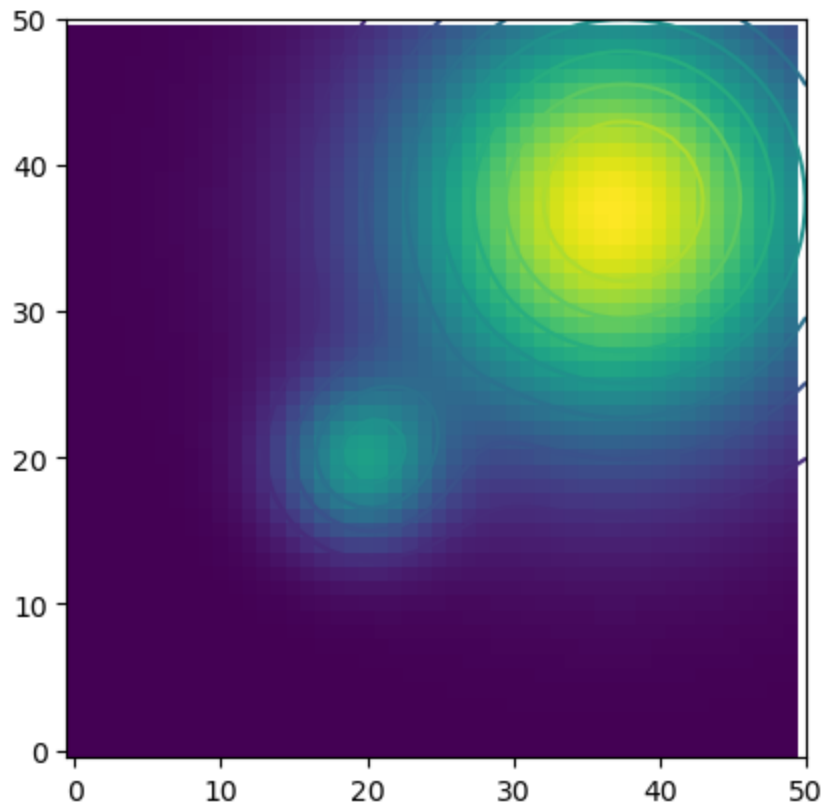
```
plt.imshow(food, origin='lower')

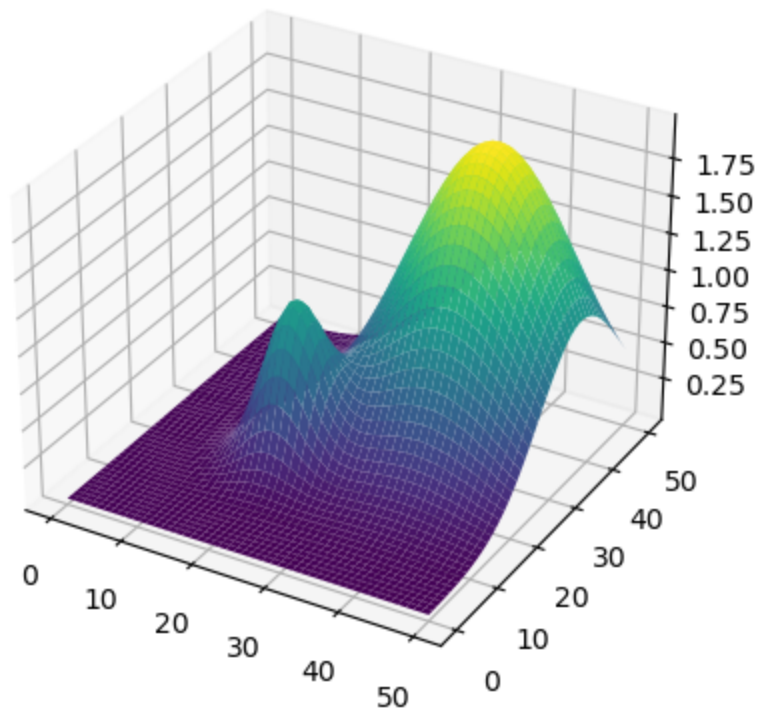
# Contour Plot
plt.contour(X, Y, food)

# 3D Surface Plot
gridx, gridy = np.meshgrid(X, Y)
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(gridx, gridy, food, cmap="viridis")
```

Out[13]: <mpl_toolkits.mplot3d.art3d.Poly3DCollection at 0x13f1376d0>

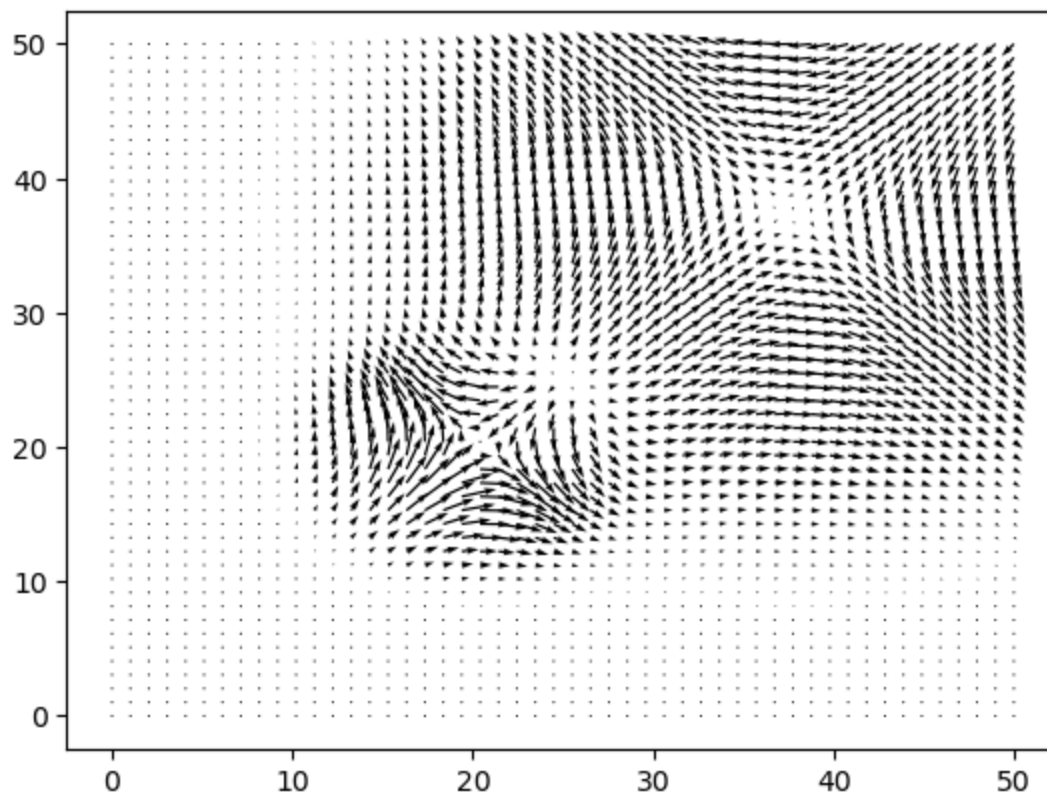




Question 2b.

```
In [15]: # Gradient Vector Field Plot
gradx, grady = np.gradient(food)
plt.quiver(X, Y, gradx, grady)
```

```
Out[15]: <matplotlib.quiver.Quiver at 0x13f876f10>
```



The gradient function takes in a values (in this food) and calculates the gradient along the x and y axis.

Question 2c.

What happens when both are low?

When both RTTP and TTRP are low, the movement it takes will often stay the same. If it is running, it will take a while for it to run/tumble again, and if it is tumbling, it will take a couple trials for it to run/tumble again.

What happens when both are high?

When both RTTP and TTRP are low, the movement it takes is more random. Instead of always choosing to run or always choosing to tumble, it will switch off randomly between the two.

What happens when one is high, the other is low?

When one is high and the other is low, it will tend to choose the one with the higher probability.

How many steps does it take?

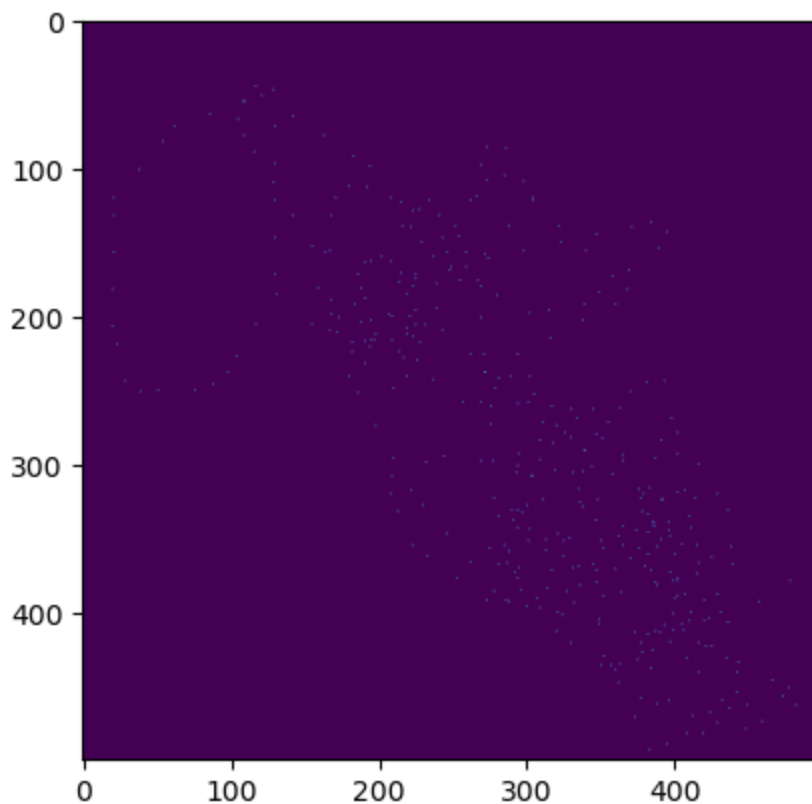
~ 50 steps

Question 2d.

```
In [16]: # Visualizing histograms with different numbers of bins
initpos = [0, 0]
foodfn = 2
rttp = 0.5
ttrp = 0.5
ntrials = 2
nsteps = 200
nbins = 500
visualize=False
verbose=False
foodfn = 2

histm, food = acoli_hist(initpos, foodfn, rttp, ttrp, ntrials, nsteps, nbins)
plt.imshow(histm)
```

Out[16]: <matplotlib.image.AxesImage at 0x13f9eaa30>



Are more bins always better?

More bins are not always better. This is because as the number of bins continues to increase, it is harder to see the trend of where the acoli goes, as the bins the acoli visits stretch out due to the increase in resolution.

Question 2e.

```
In [17]: # Initial Parameters
initpos = [0, 0]
foodfn = 1
ntrials = 100
nsteps = 200
nbins = 50
visualize=False
verbose=False

# Finding the histm and food values and putting in datatable
p_range = np.arange(0.0, 1.1, 0.1)
rows = []
for i in p_range:
    for j in p_range:
        histm, food = acolli_hist(initpos, foodfn, i, j, ntrials, nsteps, nbins)
        tot_food = np.sum(food * histm)
        avg_food = tot_food/ntrials
        row_to_add = {'RTTP and TTRP Combination': (i, j), 'Average Food Value': avg_food}
        rows.append(row_to_add)

df = pd.DataFrame(rows)
```

In [18]: df

Out[18]:

	RTTP and TTRP Combination	Average Food Value
0	(0.0, 0.0)	2.391933
1	(0.0, 0.1)	2.292439
2	(0.0, 0.2)	2.634597
3	(0.0, 0.30000000000000004)	2.288841
4	(0.0, 0.4)	2.385227
...	...	...
116	(1.0, 0.6000000000000001)	162.241929
117	(1.0, 0.7000000000000001)	162.205654
118	(1.0, 0.8)	162.941125
119	(1.0, 0.9)	160.685945
120	(1.0, 1.0)	161.154068

121 rows × 2 columns

```
In [19]: # Highest probabilities
print(df.loc[df['Average Food Value'].idxmax()])
```

```

RTTP and TTRP Combination      (0.7000000000000001, 0.0)
Average Food Value              174.100541
Name: 77, dtype: object

```

From part c, I believe that a good combination of RTTP and TTRP values to choose is one where the RTTP is high and the TTRP is 0. This is because since there is only one maxima in the image, once the a.coli reaches a maxima, the a.coli should stay in the general region of the maxima. This would allow the a.coli to consume a larger amount of food, leaving to a higher food value/trial (larger average food value).

From the combinations of RTTP and TTRP values, it is clear that the highest average food value is when the RTTP is 0.7 and the TTRP is 0, which confirms my hypothesis.

Question 2f.

I believe that different values may do better for function 2. Specifically, since there are two regions with food in the food function where one has more than the other, it may be better to have a higher probability of running. This is because once we visit a maxima, we would want to explore other maxima so that we can avoid local maxima and instead find the total maximum.

```

In [21]: # Initial Parameters
initpos = [0, 0]
ntrials = 100
nsteps = 200
nbins = 50
visualize=False
verbose=False
foodfn = 2

# Finding the histm and food values and putting in datatable
rows2 = []
for i in p_range:
    for j in p_range:
        histm, food = acolihist(initpos, foodfn, i, j, ntrials, nsteps, nbins)
        tot_food = np.sum(food * histm)
        avg_food = tot_food/ntrials
        row_to_add = {'RTTP and TTRP Combination': (i, j), 'Average Food Value': avg_food}
        rows2.append(row_to_add)

df2 = pd.DataFrame(rows2)

```

```

In [22]: df2

```

Out [22]:

RTTP and TTRP Combination		Average Food Value
0	(0.0, 0.0)	7.301490
1	(0.0, 0.1)	6.245681
2	(0.0, 0.2)	5.749658
3	(0.0, 0.30000000000000004)	6.269353
4	(0.0, 0.4)	7.266256
...	...	...
116	(1.0, 0.6000000000000001)	279.133277
117	(1.0, 0.7000000000000001)	275.153574
118	(1.0, 0.8)	269.448541
119	(1.0, 0.9)	271.706631
120	(1.0, 1.0)	278.909437

121 rows × 2 columns

In [23]:

```
# Highest probabilities
print(df2.loc[df2['Average Food Value'].idxmax()])
```

```
RTTP and TTRP Combination    (0.8, 0.1)
Average Food Value          288.581772
Name: 89, dtype: object
```

As seen in the table, the highest performing probability was where (0.8, 0.1) was RTTP and TTRP. As hypothesized, the highest performing probability has changed from part e as the TTRP is no longer zero. This makes sense as the two clusters of food are near each other, so you would want to run if you are in a local minima. If the two clusters of food were further apart, the TTRP probability may be higher as you would want to travel a longer distance.

Question 2g.

From the lecture notes, the gradient ascent algorithm dictates that positions are updated by repeatedly taking a gradient at the current position and then moving to a new position by taking a small step in the direction of the gradient. This is similar to the a.coli because the a.coli updates its behavior depending on the the change in food concentration, which can be interpreted as the "gradient". For example, when the a.coli detects it is nearing a higher food concentration, it will tumble at a smaller angle. The a.coli's strategy is different from the general gradient descent algorithm because it does not directly move in the direction of the gradient; rather, it will tumble at an angle or run in a direction.

The biologically inspired gradient ascent algorithm has an advantage because it will avoid local maxima. The vanilla gradient ascent algorithm stops when the gradient is 0. However, the biologically inspired version could run or tumble at a local maxima, which could lead it away from a local maxima and toward an absolute maxima.

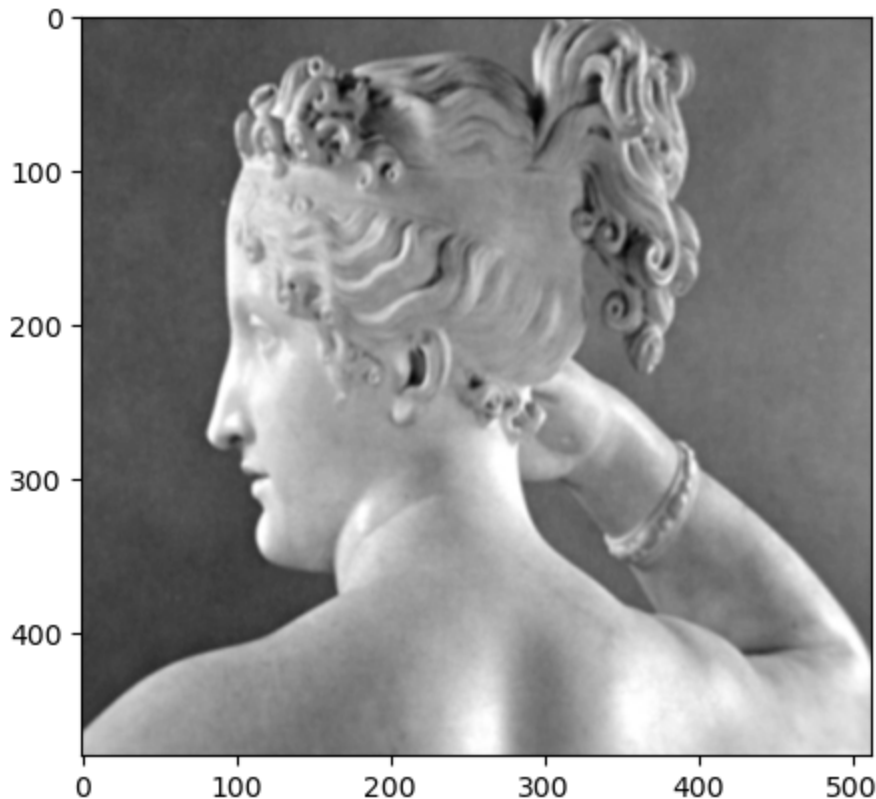
Question 3

Question 3a.

```
In [3]: img = 'Paolina.tiff'
img = np.array(Image.open(img))

img = img.astype('float')/255
plt.imshow(img, cmap='gray')
```

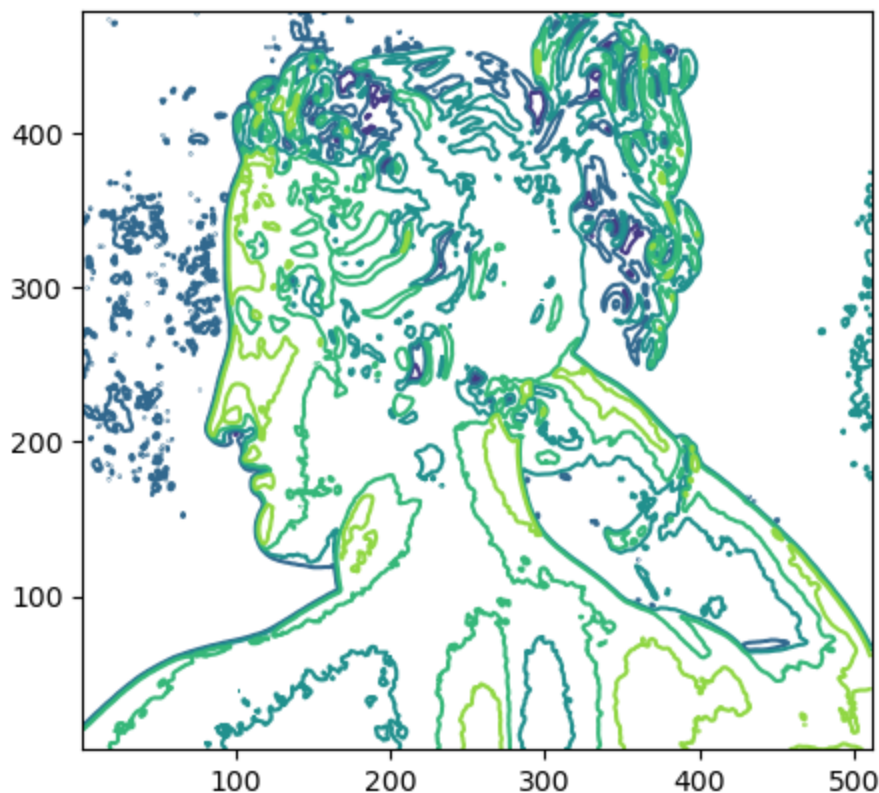
Out[3]: <matplotlib.image.AxesImage at 0x12fd02640>



Question 3b.

```
In [4]: plt.contour(img, origin='upper')
plt.axis('image')
```

Out[4]: (0.5, 511.5, 0.5, 479.5)

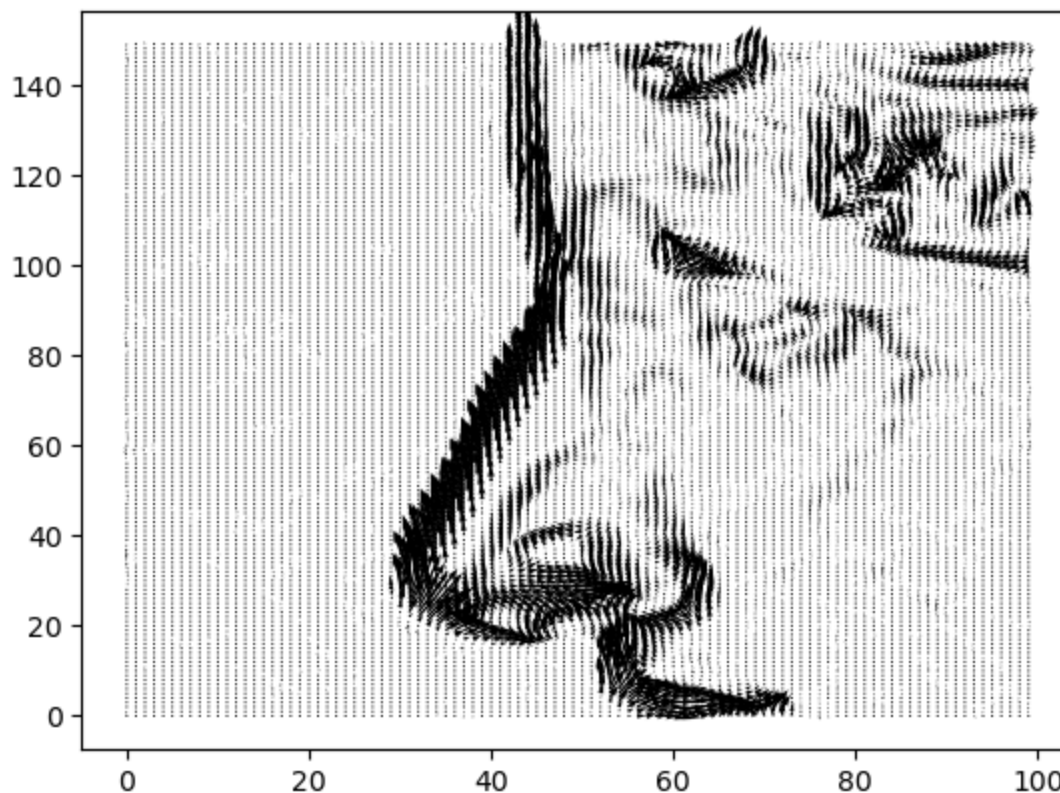


From this image, it seems like the resulting image displaying the contour in the image demonstrates the "edges" in the original input image. For example, in the original image, the edges of the woman, such as her face, the clumps of her hair, and her body, are all highlighted in the contour image, while other parts of the woman are left white. The contour function segments the parts of the image based on the "contrast" of the individual pixels to neighboring pixels. In other words, if a pixel in an image has a large difference in RGB value from its neighboring pixels, the contour plot will fill in the pixel as dark green. If a pixel has a similar color to its neighboring pixels (has a similar RGB value), then the contour plot will fill in the pixel as a lighter color (eg. white).

Question 3c.

```
In [10]: img_slice = img[150:300,50:150]
img_slice = np.flip(img_slice, axis=0)
# Gradient Vector Field Plot
gradx, grady = np.gradient(img_slice)
plt.quiver(gradx, grady)
```

```
Out[10]: <matplotlib.quiver.Quiver at 0x13e5eed60>
```

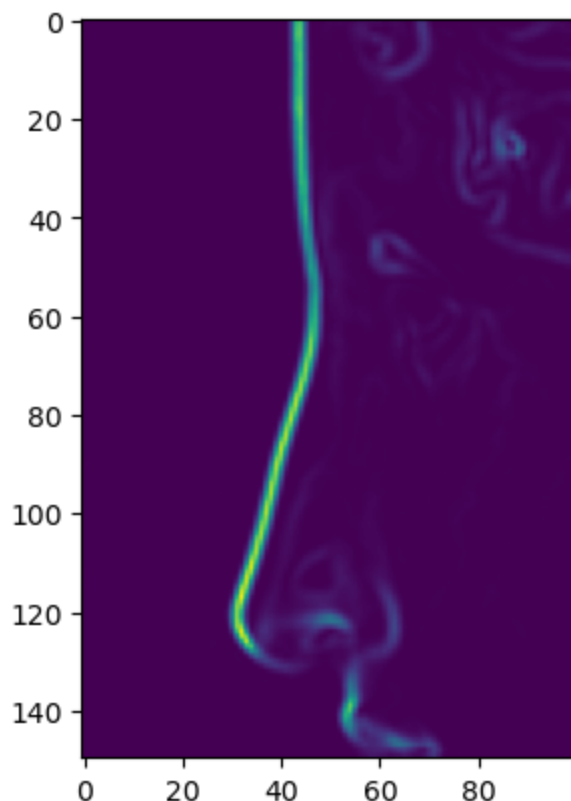


When zooming into the region of the image around the nose, it seems like the gradient vectors are pointing towards the pixels that have a larger contrast to their background. For example, from simply looking at the original image of Paulina, it is apparent that there is a large contrast between the lining of the nose and its immediate surroundings. Similarly, the gradient vector field lines to the left of the nose point right towards the nose, while the gradient vector field lines to the right of nose (on her face) point left toward her nose. Furthermore, the length of the vectors increase as they near the lining, indicating a sharper contrast in pixel value. This makes sense, as the gradients should point towards the direction of greatest change. In this case, there is a larger change in pixel value as you near the "edges" in the face.

Question 3d.

```
In [24]: gmag = np.flip((gradx ** 2 + grady ** 2) ** 1/2)
         plt.imshow(gmag)
```

```
Out[24]: <matplotlib.image.AxesImage at 0x122cef790>
```



Generally, the gradient magnitude tells us how much a function changes in the direction of the gradient. In the case of an image, the gradient magnitude tells us the greatest pixel contrast in brightness or grayscale value at the current location in the image. In the image above, these magnitudes are represented by the yellow-green highlights illuminating the face and features of Paulina. For areas of an image with a smaller gradient magnitude (smaller contrast in grayscale value), the pixels in the above image are a more dull purple.

Question 3e.

If we were to use the Paulina image as the food function, the amount of food would be dictated by the sharpness of the edges. In other words, the larger the magnitude of the gradient in a bin, the more "food" in that bin. In this case, if we started at (0,0), the *a.coli* would run and tumble toward the edges in the image. In this case, *a.coli* would tumble toward her nose, as well as the other features on her face where the magnitude of the gradient is the largest.